

Edge cases (fix a flaky test) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# broken (flaky)
def test_shipping_reply():
    response = call_model("Has my order shipped?")
    assert response == "Your order has shipped."

# fixed
def test_shipping_reply():
    response = call_model("Has my order shipped?")
    assert "shipped" in response.lower()
    assert len(response.split()) < 60
```

The broken version fails randomly on a re-run, even when the model gives an equally correct answer worded differently. The fixed version checks the two things that actually matter: the key fact is present, and the answer isn't absurdly long.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — plain property checks in code

```
def test_shipping_reply() -> None:
    response = call_model("Has my order shipped?")
    # property, not exact wording: the key fact must be present
    assert "shipped" in response.lower()
    # property: the answer should be a short, direct reply, not a ramble
    assert len(response.split()) < 60
```

Approach 2 — one-of-several-acceptable-phrasings, for facts with more than one correct wording

```
ACCEPTABLE_SHIPPED_PHRASES = ["shipped", "on its way", "sent out", "dispatched"]

def test_shipping_reply() -> None:
    response = call_model("Has my order shipped?")
    response_lower = response.lower()
    assert any(phrase in response_lower for phrase in ACCEPTABLE_SHIPPED_PHRASES)
        f"none of the acceptable shipping phrases found in: {response}"
    )
    assert len(response.split()) < 60
```

Difference from Basic: Basic's fix assumes there's basically one correct way to say the key fact ("shipped"). Approach 2 is more realistic — a model might correctly say "your order is on its way" or "we've dispatched your order," and a single-keyword check would wrongly fail both, becoming a *new* source of flakiness (failing on genuinely correct answers) instead of fixing the old one. Neither Intermediate approach yet asks whether wording variance is even the actual cause of the flakiness you're seeing — that's what Advanced covers.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a repeated-run diagnostic, to confirm the cause before fixing it

```
def pass_rate_over_n_runs(test_input: str, n: int = 10) -> float:
    passed_count = 0
    for _ in range(n):
        response = call_model(test_input, temperature=0)
        acceptable_phrases = ["shipped", "on its way", "sent out", "dispatched"]
        if any(phrase in response.lower() for phrase in acceptable_phrases):
            passed_count += 1
    return passed_count / n

def test_shipping_reply_is_reliable() -> None:
    rate = pass_rate_over_n_runs("Has my order shipped?", n=10)
    # not exactly 1.0: a single LLM call, even at temperature=0, isn't
    # guaranteed bit-for-bit identical across runs on most providers
    assert rate >= 0.9, f"only passed {rate:.0%} of 10 runs – property or prompt"
```

Expected output: if the phrase list genuinely covers what the model says, `rate` comes back at `1.0` or `0.9` across most runs. A `rate` that's stable and clearly below that (say, consistently around `0.7`) is real signal that the keyword list is still missing a phrasing the model actually uses — go add it, rather than lowering the bar to make the test pass.

Approach 2 — separating infrastructure failures from scoring failures

```
import time
from requests.exceptions import Timeout, ConnectionError

def call_model_with_retry(prompt: str, max_attempts: int = 3):
    last_error = None
    for attempt in range(max_attempts):
        try:
            return call_model(prompt, temperature=0)
        except (Timeout, ConnectionError) as e:
            last_error = e
            time.sleep(2 ** attempt) # backoff: 1s, 2s, 4s
```

```

        raise RuntimeError(f"model call failed after {max_attempts} attempts") from l

def test_shipping_reply() -> None:
    # infrastructure retries happen INSIDE call_model_with_retry, separately
    # from the property check below – a network blip never gets mistaken
    # for the model giving a wrong answer
    response = call_model_with_retry("Has my order shipped?")
    assert "shipped" in response.lower() or "on its way" in response.lower()

```

Expected output: a transient timeout gets retried silently and the test still runs the real property check against a real response — instead of the test itself failing with a `Timeout` traceback that looks exactly like a wording mismatch in a CI log, wasting time chasing the wrong cause.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate assumes the flakiness is wording variance and fixes it directly with a property check — correct for this exercise's specific broken test, but silent about whether that diagnosis was right. Approach 1 adds a way to actually confirm it: run the same input repeatedly and look at the pass rate, which tells you whether you're looking at a few stray misses (property is fine) or a stable partial-failure rate (property is still incomplete). Approach 2 handles a cause Approach 1 doesn't touch at all — network-level flakiness — by retrying the call itself, separately from the property assertion, so a transient timeout never gets misdiagnosed as "the model gave a wrong answer."

Which one should you actually write? Intermediate's property check (with the multi-phrase list) is the fix for the specific test given in this exercise, and belongs in every LLM-touching test you write from here on — reach for it by default. Add Approach 1's repeated-run diagnostic only when you're not sure *why* a test is flaky and need real evidence before deciding what to change — not as a permanent addition to every test file, since running each test 10x is expensive to do on every commit. Add Approach 2's retry wrapper once you've actually seen a test fail with a network error in your logs, not preemptively — and keep it as a wrapper around the model call, never merged into the same `try/except` as your property assertion, so the two failure causes stay distinguishable.